ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ • ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ & ΗΛΕΚΤΡΟΝΙΚΩΝ ΜΗΧΑΝΙΚΩΝ

Απόλυτο Εγχειρίδιο Ερωτήσεων & Απαντήσεων Εξεταστικής Επιτροπής

Διπλωματική Εργασία: Ανάπτυξη Ευφυών Πρακτόρων στο Παίγνιο Splendor με τον Αλγόριθμο MCTS

Επίσημο Έγγραφο Αναφοράς: `ΔιπλωματικηSplendorΤελικο(5).docx`

ΑΠΟΛΥΤΟ ΕΓΧΕΙΡΙΔΙΟ ΕΡΩΤΗΣΕΩΝ & ΑΠΑΝΤΗΣΕΩΝ ΕΞΕΤΑΣΤΙΚΗΣ ΕΠΙΤΡΟΠΗΣ

Master Defense Q&A: 45 Εξειδικευμένες Ερωτήσεις, Απαντήσεις-Παγίδες & Στρατηγική Προφορικής Υπεράσπισης

Τίτλος Διπλωματικής Εργασίας:
Ανάπτυξη Ευφυών Πρακτόρων σε Περιβάλλοντα Επιτραπέζιων Παιγνίων: Μοντελοποίηση, Υλοποίηση και Αξιολόγηση του Αλγορίθμου Monte Carlo Tree Search (MCTS) στο Παίγνιο Splendor

Ίδρυμα: Πανεπιστήμιο Δυτικής Αττικής (ΠΑΔΑ) – Σχολή Μηχανικών – Τμήμα Ηλεκτρολόγων & Ηλεκτρονικών Μηχανικών
Βασισμένο στην Τεχνική Έκθεση: `ΔιπλωματικηSplendorΤελικο(5).docx`

ΟΔΗΓΙΕΣ ΣΤΡΑΤΗΓΙΚΗΣ ΓΙΑ ΤΗΝ ΠΡΟΦΟΡΙΚΗ ΕΞΕΤΑΣΗ

1. **Δομή Απάντησης (Κανόνας 3 Επιπέδων):**
2. **Επίπεδο 1 (Άμεση Απάντηση):** Δώσε την ουσία σε 1 πρόταση (π.χ., "*Επιλέχθηκε ο MCTS διότι ο παράγοντας διακλάδωσης $b > 35$ καθιστά τον Minimax υπολογιστικά ανεφάρμοστο*").
3. **Επίπεδο 2 (Μαθηματική/Τεχνική Τεκμηρίωση):** Ανάλυσε τον τύπο, τον αλγόριθμο ή τη δομή δεδομένων (π.χ., ανάλυση UCB1, 10^{81} καταστάσεις, Shannon Type B).
4. **Επίπεδο 3 (Πειραματική Επιβεβαίωση από την Εργασία σου):** Σύνδεσε την απάντηση με τα γραφήματά σου (π.χ., "*Αυτό επιβεβαιώνεται στο Σχήμα 22 του Thinking Effort και στο 80.1% ποσοστό νικών*").
5. **Αναγνώριση του Προφίλ των Εξεταστών:**
6. **Ο Θεωρητικός Καθηγητής:** Ενδιαφέρεται για MDPs, ανισότητα Hoeffding, UCB1, Markov property.

7. **Ο Προγραμματιστής Καθηγητής:** Ενδιαφέρεται για Python GIL, State Cloning $20\times$, Pruning, memory footprint, OOP/MVC.
8. **Ο Στατιστικολόγος / Reviewer:** Ενδιαφέρεται για τα νούμερα των 251 παρτίδων, το rolling win rate, το p-value και το αν ο Heuristic Agent είχε bugs.

ΠΙΝΑΚΑΣ ΠΕΡΙΕΧΟΜΕΝΩΝ ΕΡΩΤΗΣΕΩΝ

- ΕΝΟΤΗΤΑ 1: Θεωρία Παιγνίων, MDP & Μαθηματική Πολυπλοκότητα (Ερωτήσεις 1–10)
- ΕΝΟΤΗΤΑ 2: Αλγόριθμος MCTS, UCT & Υπερπαράμετροι (Ερωτήσεις 11–20)
- ΕΝΟΤΗΤΑ 3: Αρχιτεκτονική Λογισμικού, Κώδικας & Βελτιστοποίηση Python (Ερωτήσεις 21–30)
- ΕΝΟΤΗΤΑ 4: Πειραματικά Αποτελέσματα, Στατιστική & Ερωτήσεις-Παγίδες (Ερωτήσεις 31–40)
- ΕΝΟΤΗΤΑ 5: Αυτοκριτική, Περιορισμοί & Μελλοντική Έρευνα (Ερωτήσεις 41–45)

ΕΝΟΤΗΤΑ 1: Θεωρία Παιγνίων, MDP & Μαθηματική Πολυπλοκότητα

Ερώτηση 1: Γιατί επιλέξατε το Splendor ως πεδίο δοκιμών για τη διπλωματική σας και όχι ένα πιο γνωστό παιχνίδι όπως το Σκάκι ή η Ντάμα;

Υποδειγματική Απάντηση:
Το Σκάκι και η Ντάμα είναι παιχνίδια **ντετερμινιστικά και τέλειας πληροφόρησης**, τα οποία έχουν ήδη επιλυθεί σε υπεράνθρωπο επίπεδο μέσω εξαντλητικής αναζήτησης ή βαθιών δικτύων. Το Splendor επιλέχθηκε διότι αντιπροσωπεύει μία πολύ πιο ρεαλιστική κατηγορία προβλημάτων του πραγματικού κόσμου: 1. Συνδυάζει **διαχείριση περιορισμένων πόρων (Resource Management)** με ανώτατο όριο 10 μαρκών. 2. Εμπεριέχει **στοχαστικότητα και ατελή πληροφόρηση**, καθώς οι νέες κάρτες που ανοίγουν από την τράπουλα είναι άγνωστες εκ των προτέρων. 3. Επιβάλλει την ανάγκη **Αναβαλλόμενης Ικανοποίησης (Delayed Gratification)** μέσω του Engine Building, όπου η μυωπική αναζήτηση άμεσων πόντων οδηγεί σε βέβαιη ήττα.

Ερώτηση 2: Πώς ταξινομείται το Splendor αυστηρά στο πλαίσιο της Θεωρίας Παιγνίων;

Υποδειγματική Απάντηση:
Βάσει της Θεωρίας Παιγνίων, το Splendor ταξινομείται ως: * **Παίγνιο 2 Παικτών (2-Player Game)** στην πειραματική μας διάταξη. * **Μηδενικού Αθροίσματος (Strictly Competitive / Zero-Sum):** Το κέρδος του ενός ισούται με την απώλεια του άλλου (payoff $+1.0$ για νίκη, 0.0 για ήττα). * **Τέλειας Πληροφορίας στο Ταμπλό (Perfect Information on Board):** Όλα τα ανοιχτά στοιχεία (κάρτες, μάρκες, ευγενείς) είναι πλήρως ορατά και στους δύο παίκτες. * **Στοχαστικό στο Τράβηγμα (Stochastic in Deck Draw):** Υπάρχει τυχαιότητα στη σειρά εμφάνισης των νέων καρτών από την τράπουλα. * **Σειριακό / Διακριτών Γύρων (Turn-based):** Οι παίκτες ενεργούν εναλλάξ χωρίς ταυτόχρονες κινήσεις.

Ερώτηση 3: Τι είναι η Μαρκοβιανή Ιδιότητα (Markov Property) και πώς ακριβώς αξιοποιήθηκε στην υλοποίηση του συστήματός σας;

Υποδειγματική Απάντηση:
Η Μαρκοβιανή Ιδιότητα ορίζει ότι η πιθανότητα μετάβασης στην επόμενη κατάσταση S_{t+1} εξαρτάται **αποκλειστικά και μόνο** από την τρέχουσα κατάσταση S_t και την επιλεγμένη ενέργεια A_t:

$$\mathbb{P}(S_{t+1} = s' \mid S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, \dots) = \mathbb{P}(S_{t+1} = s' \mid S_t = s_t, A_t = a_t)$$

Δηλαδή, **το ιστορικό των προηγούμενων κινήσεων είναι αδιάφορο**.
Στην εργασία μας, η ιδιότητα αυτή υπήρξε θεμελιώδης για την υλοποίηση της **Μόνιμης Βάσης Γνώσης (Persistent Knowledge Base)**: μας επέτρεψε να κωδικοποιήσουμε κάθε ταμπλό με ένα μοναδικό hash

(`state_hash`). Αν δύο εντελώς διαφορετικές παρτίδες καταλήξουν στο ίδιο ταμπλό, ο MCTS ανακτά άμεσα την αποθηκευμένη εμπειρία, ανεξάρτητα από το πώς φτάσαμε εκεί!

Ερώτηση 4: Γιατί αποτυγχάνει ο κλασικός αλγόριθμος Minimax (με Alpha-Beta Pruning) στο Splendor; Ποια είναι η πολυπλοκότητα του χώρου καταστάσεων;

Υποδειγματική Απάντηση:

Ο Minimax αποτυγχάνει λόγω της **Συνδυαστικής Έκρηξης (Combinatorial Explosion)**. * Ο μέσος παράγοντας διακλάδωσης στο Splendor είναι $b \approx 20$ έως 45 νόμιμες κινήσεις ανά γύρο. * Το μέσο βάθος μιας παρτίδας είναι $d \approx 50$ έως 60 ημι-γύροι. * Το συνολικό μέγεθος του δέντρου παιγνίου εκτιμάται σε:

$$|\mathcal{T}| = b^d \approx 30^{55} \approx 10^{81} \text{ καταστάσεις!}$$

Ο αριθμός 10^{81} ξεπερνά όλα τα άτομα του παρατηρήσιμου σύμπαντος ($\approx 10^{80}$). Ο Minimax αναπτύσσει το δέντρο συμμετρικά και εξαντλητικά σε σταθερό βάθος, με αποτέλεσμα στα 2–3 δευτερόλεπτα να μπορεί να φτάσει μόλις σε βάθος $d = 3$, μέγεθος παντελώς ανεπαρκές για στρατηγικό σχεδιασμό. Αντίθετα, ο MCTS αναπτύσσει το δέντρο **ασύμμετρα (Asymmetric Tree Growth)**, εστιάζοντας μόνο στα κλαδιά υψηλής απόδοσης.

Ερώτηση 5: Τι ήταν η θεωρητική διάκριση του Claude Shannon μεταξύ Type A και Type B στρατηγικών, και πού κατατάσσεται η εργασία σας;

Υποδειγματική Απάντηση:

Το 1950, ο Claude Shannon πρότεινε δύο προσεγγίσεις για το σκάκι: * **Type A (Brute-Force):** Εξαντλητική εξέταση όλων των πιθανών κινήσεων μέχρι ένα αυστηρό βάθος (όπως έκανε ο Deep Blue). * **Type B (Selective Search):** Επιλεκτική αναζήτηση μόνο σε κινήσεις που κρίνονται στρατηγικά λογικές, αγνοώντας τις παράλογες διακλαδώσεις.

Η διπλωματική μας εργασία κατατάσσεται ξεκάθαρα στην κατηγορία Shannon Type B: Ο MCTS, σε συνδυασμό με τον μηχανισμό **Action Pruning**, αποκόπτει προκαταβολικά τις ασύμφορες κινήσεις και ερευνά βαθιά μόνο τις υποσχόμενες.

Ερώτηση 6: Πώς διαχειρίζεται ο αλγόριθμος τη στοχαστικότητα και τις κρυφές κάρτες της τράπουλας; Τι είναι η τεχνική Determinization;

Υποδειγματική Απάντηση:

Εφαρμόζουμε την τεχνική του **Καθορισμού (Determinization)**:

Σε κάθε προσομοίωση (rollout), το σύστημα δημιουργεί ένα υποθετικό αντίγραφο της τράπουλας, ανακατεύοντας τυχαία τις κλειστές κάρτες που απομένουν. Με αυτόν τον τρόπο, το στοχαστικό παίγνιο μετατρέπεται προσωρινά σε ντετερμινιστικό για τη διάρκεια της συγκεκριμένης προσομοίωσης. Επειδή εκτελούνται εκατοντάδες τέτοια rollouts σε διαφορετικά "τυχαία σύμπαντα", η τυχαιότητα εξομαλύνεται στατιστικά (Monte Carlo averaging) και ο πράκτορας λαμβάνει αποφάσεις ανεξάρτητες από τυχαίες διακυμάνσεις.

Ερώτηση 7: Ποια είναι η διαφορά μεταξύ Τέλειας Πληροφορίας (Perfect Information) και Πλήρους Πληροφορίας (Complete Information);

Υποδειγματική Απάντηση:
*Πλήρης Πληροφορία (Complete Information): Όλοι οι παίκτες γνωρίζουν πλήρως τους κανόνες, τις συναρτήσεις αμοιβής (payoffs) και τις δυνατές στρατηγικές των αντιπάλων. Το Splendor είναι παίγνιο πλήρους πληροφορίας. * Τέλεια Πληροφορία (Perfect Information): Οι παίκτες γνωρίζουν ανά πάσα στιγμή το πλήρες ιστορικό και την ακριβή κατάσταση του συστήματος χωρίς κρυφά στοιχεία. Στο Splendor, το ταμπλό είναι τέλειας πληροφορίας, αλλά η κλειστή τράπουλα εισάγει στοιχείο ατελούς πληροφορίας*, το οποίο επιλύεται μέσω του Determinization.

Ερώτηση 8: Πώς συνδέεται η προσέγγισή σας με την αρχιτεκτονική του AlphaGo και του AlphaZero της DeepMind;

Υποδειγματική Απάντηση:
Ο AlphaZero αντικατέστησε τα τυχαία rollouts και τη χειροκίνητη ευρετική συνάρτηση του MCTS με δύο Βαθιά Νευρωνικά Δίκτυα: 1. **Policy Network:** Προβλέπει τις καλύτερες κινήσεις (αντίστοιχο του δικού μας Action Pruning & Heuristic Selection). 2. **Value Network:** Αξιολογεί την κατάσταση του ταμπλό (αντίστοιχο της δικής μας συνάρτησης `_evaluate_position`).
Η εργασία μας αποτελεί το **θεμελιώδες αλγοριθμικό υπόβαθρο** πάνω στο οποίο χτίστηκε ο AlphaZero, αποδεικνύοντας τη δύναμη του MCTS ακόμη και με κλασικές ευρετικές συναρτήσεις πριν την εισαγωγή των νευρωνικών δικτύων.

Ερώτηση 9: Γιατί δεν χρησιμοποιήσατε κλασικό Reinforcement Learning (π.χ. Q-Learning);

Υποδειγματική Απάντηση:
Το απλό tabular Q-Learning απαιτεί έναν πίνακα $Q(s, a)$ για κάθε ζεύγος κατάστασης-ενέργειας. Δεδομένου ότι ο χώρος καταστάσεων είναι 10^{81}, ο πίνακας θα απαιτούσε άπειρη μνήμη (Curse of Dimensionality) και δεν θα συνέκλινε ποτέ. Το Deep Q-Learning (DQN) θα μπορούσε να χρησιμοποιηθεί, αλλά απαιτεί τεράστια υπολογιστική ισχύ εκπαίδευσης (GPUs) και είναι ευάλωτο σε αστάθειες. Ο MCTS με Knowledge Base προσέφερε άμεση, online λήψη αποφάσεων με μηδενική προεκπαίδευση.

Ερώτηση 10: Τι είναι το φαινόμενο του Ορίζοντα (Horizon Effect) και πώς το αντιμετωπίζει ο MCTS;

Υποδειγματική Απάντηση:
Το Horizon Effect συμβαίνει όταν ένας αλγόριθμος σταθερού βάθους (όπως ο Minimax σε βάθος $d = 4$) αναβάλλει μία αναπόφευκτη καταστροφή σπρώχνοντάς την πέρα από το οπτικό του πεδίο (horizon). Ο MCTS το εξαλείφει, διότι τα rollouts φτάνουν μέχρι το τέλος της παρτίδας ή σε μεγάλο βάθος ($d = 30$), "βλέποντας" τις μακροπρόθεσμες συνέπειες μιας κίνησης.

ΕΝΟΤΗΤΑ 2: Αλγόριθμος MCTS, UCT & Υπερπαράμετροι

Ερώτηση 11: Εξηγήστε αναλυτικά τα τέσσερα (4) στάδια του αλγορίθμου MCTS.

Υποδειγματική Απάντηση:
Ο αλγόριθμος εκτελείται επαναληπτικά εντός του διαθέσιμου χρόνου (2–3 sec) μέσω 4 φάσεων: 1. **Selection (Επιλογή):** Ξεκινώντας από τη ρίζα, κατεβαίνει στο δέντρο επιλέγοντας σε κάθε διακλάδωση το παιδί με το μέγιστο σκορ **UCB1**, μέχρι να φτάσει σε έναν κόμβο που δεν έχει αναπτυχθεί πλήρως. 2. **Expansion (Επέκταση):** Επιλέγει μία μη δοκιμασμένη ενέργεια από τις `untried_actions` (αφού περάσουν από το **Action Pruning**) και δημιουργεί έναν νέο κόμβο-παιδί. 3. **Simulation / Rollout (Προσομοίωση):** Από τον νέο κόμβο εκτελείται γρήγορη προσομοίωση μέχρι βάθος $d_{max} = 30$ γύρων, με ημι-ευρετική επιλογή κινήσεων (`_select_smart_action`). 4. **Backpropagation (Οπισθοδιάδοση):** Το αποτέλεσμα $R \in [0.0, 1.0]$ της αξιολόγησης μεταδίδεται προς τα πάνω, ενημερώνοντας όλους τους προγονικούς κόμβους: $N_i \leftarrow N_i + 1$ και $W_i \leftarrow W_i + R$.

Ερώτηση 12: Αναλύστε μαθηματικά τον τύπο UCB1 / UCT όρο προς όρο.

Υποδειγματική Απάντηση:
Ο τύπος UCB1 είναι:

$$UCT(i) = \frac{W_i}{n_i} + C \cdot \sqrt{\frac{\ln N}{n_i}}$$

* $\frac{W_i}{n_i}$ (**Exploitation Term**): Η μέση τιμή επιτυχίας του κόμβου i (win rate). Μεγιστοποιεί την αξιοποίηση των ήδη γνωστών κερδοφόρων κινήσεων. * $C \cdot \sqrt{\frac{\ln N}{n_i}}$ (**Exploration Term**): Ο όρος εξερεύνησης. Όταν ένας κόμβος έχει λίγες επισκέψεις ($n_i \ll N$), ο παρονομαστής είναι μικρός και ο όρος εκτινάσσεται, αναγκάζοντας τον αλγόριθμο να δοκιμάσει υποσχόμενα ανεξερεύνητα μονοπάτια. * C: Η σταθερά εξισορρόπησης των δύο όρων.

Ερώτηση 13: Πώς ακριβώς προκύπτει η τιμή $C_{base} = \sqrt{2} \approx 1.414$;

Υποδειγματική Απάντηση:
Η τιμή $\sqrt{2}$ προκύπτει θεωρητικά από την εφαρμογή της **Ανισότητας Hoeffding-Chernoff** στο πρόβλημα των Πολυβραχιόνιων Ληστών (Multi-Armed Bandits) από τους Auer, Cesa-Bianchi και Fischer (2002). Αποδεικνύεται μαθηματικά ότι με $C = \sqrt{2}$, το συσσωρευμένο διαφυγόν κέρδος (cumulative regret) αυξάνεται μόνο **λογαριθμικά με τον χρόνο**, που είναι το θεωρητικά βέλτιστο εφικτό άνω όριο.

Ερώτηση 14: Τι είναι το Adaptive UCT και ποια είναι η λογική πίσω από τα κατώφλια $N_{100} = 100$ και $N_{200} = 200$;

Υποδειγματική Απάντηση:

Είναι μία από τις κύριες καινοτομίες της εργασίας μας. Στον κλασικό MCTS, η σταθερά C παραμένει σταθερή. Στο δικό μας σύστημα: * Όταν οι επισκέψεις ενός κόμβου υπερβούν τις **100 (N_{100})**, το C μειώνεται σε $C \times 0.60$. * Όταν υπερβούν τις **200 (N_{200})**, το C μειώνεται σε $C \times 0.50$.

Αιτιολόγηση: Στις 200 επισκέψεις έχουμε αποκτήσει **Στατιστική Βεβαιότητα (Statistical Confidence)** για την αξία της κίνησης. Η συνέχιση της εξερεύνησης αποτελεί σπατάλη του αυστηρού χρονικού προϋπολογισμού (2–3 sec). Μειώνοντας το C, εξαναγκάζουμε τον αλγόριθμο να στραφεί στο **επιθετικό Exploitation**, εμβαθύνοντας αποκλειστικά στο σίγουρο νικηφόρο μονοπάτι.

Ερώτηση 15: Γιατί κατά την επιλογή της τελικής κίνησης επιλέγετε το "Robust Child" ($\arg\max N$) και όχι τον κόμβο με το μέγιστο Win Rate;

Υποδειγματική Απάντηση:

Αυτή είναι κλασική ερώτηση-παγίδα!

Αν επιλέγαμε το μέγιστο win rate (W/n), ένας κόμβος που δοκιμάστηκε μόλις 1 φορά και έτυχε να κερδίσει θα είχε win rate 1.0 (100%), αλλά μηδενική στατιστική αξιοπιστία!

Αντίθετα, ο πιο επισκεφθείς κόμβος ($\arg\max N$) έχει δοκιμαστεί εκατοντάδες φορές. Επειδή ο τύπος UCB1 κατευθύνει φυσικά τη συντριπτική πλειονότητα των επισκέψεων στα αποδεδειγμένα ανώτερα κλαδιά, ο κόμβος με τις περισσότερες επισκέψεις είναι **στατιστικά ο πλέον εύρωστος και ασφαλής (Robust Child)**.

Ερώτηση 16: Πώς λειτουργεί ο μηχανισμός Action Pruning και πώς είστε βέβαιος ότι δεν αποκόπτει τη βέλτιστη κίνηση;

Υποδειγματική Απάντηση:

Το Pruning (`_prune_actions`) εφαρμόζει τρεις αυστηρούς κανόνες επικράτειας: 1. **Κανόνας 10 Μαρκών:** Αποκόπτει το Reserve αν ο παίκτης έχει 10 μάρκες, διότι η υποχρεωτική απόρριψη μάρκας αποτελεί μαθηματική απώλεια τέμπο. 2. **Early Tier 3 Reserve:** Αποκόπτει το Reserve καρτών Tier 3 στους πρώτους 15 γύρους αν τα μόνιμα μπόνους είναι < 3, αποτρέποντας το μπλοκάρισμα των θέσεων κράτησης με ανέφικτες κάρτες. 3. **Saturation Pruning:** Αποκόπτει τη λήψη 2 ίδιων μαρκών αν ο παίκτης έχει ήδη ≥ 8 μάρκες και υπάρχουν άμεσες αγορές.

Διασφάλιση: Οι κανόνες αποκόπτουν μόνο ενέργειες που είναι **αυστηρά υποδεέστερες (strictly dominated)**. Επιπλέον, υπάρχει δικλείδα ασφαλείας: αν το pruning αδειάσει τη λίστα, επιστρέφει αυτόματα όλες τις αρχικές κινήσεις.

Ερώτηση 17: Γιατί θέσατε όριο βάθους $d_{max} = 30$ γύρων στο Rollout;

Υποδειγματική Απάντηση:

Χωρίς όριο βάθους, δύο τυχαίοι πράκτορες μπορεί να περιφέρονται άσκοπα για 80+ γύρους μέχρι να κερδίσει κάποιος κατά τύχη. Αυτό θα εκτόξευε τη χρονική διάρκεια του rollout, μειώνοντας τον αριθμό

των προσομοιώσεων σε < 100 ανά κίνηση. Με το όριο $d_{max} = 30$, κόβουμε έγκαιρα το rollout και αξιολογούμε τη θέση με τη συνάρτηση `_evaluate_position`, επιτυγχάνοντας **1.500+ rollouts ανά κίνηση**!

Ερώτηση 18: Αναλύστε τη συνάρτηση αξιολόγησης `_evaluate_position`. Γιατί δώσατε βάρος 80% στους πόντους και 12% στα μπόνους;

Υποδειγματική Απάντηση:
Η συνάρτηση είναι:

$$Score = \left(\frac{\Delta \text{Points}}{20.0} \times 0.80 \right) + \left(\frac{\Delta \text{Bonuses}}{12.0} \times 0.12 \right) + \text{ProximityBonus}$$

* **80% στους Πόντους ($W_{points} = 0.80$):** Οι πόντοι αποτελούν τον τελικό στόχο νίκης (15 VP). * **12% στα Μπόνους ($W_{engine} = 0.12$): Αυτό είναι το στρατηγικό κλειδί!** Στους πρώτους 15 γύρους, κανένας παίκτης δεν έχει πόντους ($\Delta \text{Points} = 0$). Εκεί, το 12% των μπόνους καθορίζει το 100% **της απόφασης**, αναγκάζοντας τον MCTS να επενδύσει σε μόνιμες εκπτώσεις (Engine Building) αντί να περιφέρεται άσκοπα!

Ερώτηση 19: Τι είναι το Proximity Bonus και γιατί η κλιμάκωσή του είναι εκθετική;

Υποδειγματική Απάντηση:
Το Proximity Bonus πριμοδοτεί τον πράκτορα όταν πλησιάζει το όριο των 15 πόντων: * ≥ 8 πόντους: $+0.08$ * ≥ 10 πόντους: $+0.15$ * ≥ 12 πόντους: $+0.25$ * ≥ 14 πόντους: $+0.35$
Η κλιμάκωση είναι εκθετική διότι στους 14 πόντους ο παίκτης απέχει μόλις 1 πόντο από τη νίκη. Το τεράστιο μπόνους $+0.35$ δημιουργεί **"killer instinct" (ένστικτο τελειώματος)**, αποτρέποντας τον πράκτορα από το να συνεχίσει να χτίζει υποδομές όταν μπορεί να τελειώσει αμέσως την παρτίδα!

Ερώτηση 20: Γιατί θέσατε κατώφλι $N_{min_kb} \geq 5$ επισκέψεων για την αποθήκευση στη Βάση Γνώσης;

Υποδειγματική Απάντηση:
Για την πλήρη αποτροπή του φαινομένου **Data Poisoning (Μόλυνση Δεδομένων)**. Αν αποθηκεύαμε κόμβους με 1 ή 2 επισκέψεις, μία εντελώς τυχαία ή κακή κίνηση που έτυχε να κερδίσει κατά λάθος σε ένα rollout θα αποθηκευόταν ως "άριστη" στη μνήμη. Το όριο των 5 επισκέψεων λειτουργεί ως στατιστικό φίλτρο, εξασφαλίζοντας ότι μόνο δοκιμασμένες κινήσεις εισάγονται στη μόνιμη γνώση.

ΕΝΟΤΗΤΑ 3: Αρχιτεκτονική Λογισμικού, Κώδικας & Βελτιστοποίηση Python

Ερώτηση 21: Ποιο ήταν το μεγαλύτερο τεχνικό/προγραμματιστικό πρόβλημα που αντιμετωπίσατε και πώς το επιλύσατε;

Υποδειγματική Απάντηση:
Το μεγαλύτερο bottleneck ήταν το **State Cloning** κατά τη διάρκεια των χιλιάδων rollouts. Αρχικά, χρησιμοποιήσαμε την ενσωματωμένη συνάρτηση `copy.deepcopy()` της Python. Λόγω αναδρομικής επιθεώρησης τύπων (reflection), ήταν απαγορευτικά αργή, περιορίζοντας τον MCTS σε μόλις **80 προσομοιώσεις/sec**.
Υλοποιήσαμε τη δική μας μέθοδο `get_state_copy()`, η οποία αντιγράφει ρητά μόνο τα απολύτως απαραίτητα primitives (ρηχά λεξικά, λίστες ακεραίων). Αυτό επέφερε **επιτάχυνση άνω του** $20\times$, εκτοξεύοντας τον ρυθμό σε **1.500+ rollouts/sec**, επιτρέποντας στον πράκτορα να λειτουργεί σε πραγματικό χρόνο!

Ερώτηση 22: Γιατί επιλέξατε την Python και όχι μια μεταγλωττιζόμενη γλώσσα όπως η C++ ή η Rust που είναι πολύ ταχύτερες;

Υποδειγματική Απάντηση:
Αν και η C++ θα προσέφερε ταχύτερη εκτέλεση, η Python επιλέχθηκε στρατηγικά διότι: 1. Προσφέρει εξαιρετική ταχύτητα ανάπτυξης (Rapid Prototyping) και καθαρή αντικειμενοστραφή δομή. 2. Διαθέτει το κορυφαίο οικοσύστημα ανάλυσης δεδομένων και οπτικοποίησης στον κόσμο (**Matplotlib, NumPy, Pygame**).
Χωρίς αυτά τα εργαλεία, θα ήταν αδύνατο να κατασκευαστεί το **Deep Dive Analytics Dashboard** και να εξαχθούν τα 4 γραφήματα τηλεμετρίας, τα οποία αποτελούν τον βασικό ερευνητικό καρπό της διπλωματικής.

Ερώτηση 23: Τι είναι το Global Interpreter Lock (GIL) της Python και πώς επηρέασε την εργασία σας;

Υποδειγματική Απάντηση:
Το GIL είναι ένας μηχανισμός συγχρονισμού του CPython που επιτρέπει σε ένα μόνο native thread να εκτελεί bytecode Python κάθε φορά, ακόμη κι αν υπάρχουν πολλαπλοί πυρήνες CPU. Αυτό μας εμπόδισε να υλοποιήσουμε πραγματικό παράλληλο MCTS (Multi-threaded Root Parallelization). Αντισταθμίσαμε αυτόν τον περιορισμό μέσω του custom state cloning και του Action Pruning, μεγιστοποιώντας την αποδοτικότητα σε έναν πυρήνα.

Ερώτηση 24: Πώς εφαρμόστηκε το αρχιτεκτονικό πρότυπο MVC στην υλοποίησή σας;

Υποδειγματική Απάντηση:

*Model (`game_engine/`): Περιλαμβάνει τους κανόνες του παιχνιδιού, την κλάση `GameState` και τις ενέργειες, εντελώς απομονωμένα από γραφικά. * View (`gui/`, `analytics/`): Η γραφική διεπαφή Pygame και τα εργαλεία οπτικοποίησης των γραφημάτων. * Controller (`main.py`, `agents/`): Ελέγχει τη ροή του αγώνα, τα χρονόμετρα, καλεί τους πράκτορες και εφαρμόζει τις κινήσεις στο Model.*

*Αυτή η αποσύνδεση επέτρεψε την απρόσκοπτη εκτέλεση 251 headless παρτίδων** χωρίς γραφικά, σε ελάχιστο χρόνο!

Ερώτηση 25: Πώς υλοποιήθηκε το State Hashing και πώς αποφεύγονται οι συγκρούσεις (Hash Collisions);

Υποδειγματική Απάντηση:

Η μέθοδος `get_hash()` δημιουργεί μία κανονικοποιημένη συμβολοσειρά (string) που περιλαμβάνει: * Ταξινομημένες τις μάρκες των παικτών και της τράπεζας. * Ταξινομημένες τις διαθέσιμες κάρτες ανά Tier (με τα IDs τους). * Τους πόντους και τα μόνιμα μπόνους.

Η συμβολοσειρά περνάει από αλγόριθμο κατακερματισμού (MD5-like 128-bit hash). Η πιθανότητα σύγκρουσης σε δείγμα 80.000 καταστάσεων είναι μικρότερη από 10^{-15}, καθιστώντας την αναπαράσταση απόλυτα ασφαλή.

Ερώτηση 26: Πώς λειτουργεί το σύστημα Τηλεμετρίας (Logger) χωρίς να καθυστερεί την εκτέλεση του MCTS;

Υποδειγματική Απάντηση:

Το σύστημα `analytics/recorder.py` καταγράφει μεταδεδομένα **ασύγχρονα στο τέλος κάθε γύρου** και όχι μέσα στη λούπα των rollouts. Συλλέγει μόνο τα συνοπτικά μεγέθη της ρίζας του MCTS (αριθμός simulations, βάθος, εκτιμώμενο win probability), αποθηκεύοντας τα δεδομένα στη μνήμη RAM και γράφοντάς τα στο αρχείο `games.json` μόνο μετά την ολοκλήρωση της παρτίδας.

Ερώτηση 27: Πώς υπολογίστηκε η ποιότητα των κινήσεων στο Move Quality Strip Plot;

Υποδειγματική Απάντηση:

Για κάθε κίνηση που εκτελείται στο ταμπλό: 1. Το σύστημα παγώνει το ταμπλό και εκτελεί μία βαθιά αξιολόγηση MCTS για να εντοπίσει τη μαθηματικά βέλτιστη κίνηση a_{best} με πιθανότητα νίκης P_{best}. 2. Υπολογίζει την πιθανότητα νίκης P_{actual} της κίνησης που επέλεξε ο παίκτης. 3. Υπολογίζει τη διαφορά $\Delta P = P_{best} - P_{actual}$. 4. Ταξινομεί την κίνηση: **Best Move** ($\Delta P = 0$), **Inaccuracy** ($\Delta P < 10\%$), **Mistake** ($10\% \leq \Delta P < 20\%$), **Blunder** ($\Delta P \geq 20\%$).

Ερώτηση 28: Ποιες βιβλιοθήκες τρίτων χρησιμοποιήθηκαν και ποιος ο ρόλος τους;

Υποδειγματική Απάντηση:
*Pygame: Ανάπτυξη του GUI για ζωντανή παρακολούθηση και ανθρώπινο έλεγχο. * Matplotlib: Παραγωγή των 4 γραφημάτων της μεμονωμένης παρτίδας και του dashboard των συνολικών αποτελεσμάτων. * NumPy: Επεξεργασία πινάκων δεδομένων και υπολογισμός του κινητού μέσου όρου (Rolling Average). * Όλος ο πυρήνας της μηχανής του Splendor και οι αλγόριθμοι MCTS / Heuristic υλοποιήθηκαν απολύτως από το μηδέν (from scratch)* σε αμιγή Python!

Ερώτηση 29: Υπάρχει κίνδυνος διαρροής μνήμης (Memory Leak) κατά την αποθήκευση 80.000 καταστάσεων;

Υποδειγματική Απάντηση:
Όχι. Η μνήμη της Βάσης Γνώσης είναι ένα απλό Python dictionary με 80.000 εγγραφές. Κάθε εγγραφή περιλαμβάνει μόνο δύο αριθμούς (`visits`: int, `value`: float), καταλαμβάνοντας περίπου **80–90 MB RAM**, μέγεθος απολύτως αμελητέο για τα σύγχρονα υπολογιστικά συστήματα. Επιπλέον, στο τέλος κάθε παρτίδας το δέντρο αναζήτησης του MCTS διαγράφεται πλήρως από τον Garbage Collector της Python.

Ερώτηση 30: Πώς εξασφαλίσατε την αναπαραγωγιμότητα (Reproducibility) των πειραμάτων σας;

Υποδειγματική Απάντηση:
Όλες οι παρτίδες κατέγραψαν το πλήρες ιστορικό τους στο δομημένο αρχείο `data/games.json`. Επιπλέον, για τις μεμονωμένες παρτίδες ελέγχου (όπως η Game 114), χρησιμοποιήθηκαν σταθερά seeds στην ψευδοτυχαία γεννήτρια αριθμών της Python (`random.seed`), επιτρέποντας την ακριβή επανάληψη της παρτίδας κίνηση-προς-κίνηση.

ΕΝΟΤΗΤΑ 4: Πειραματικά Αποτελέσματα, Στατιστική & Ερωτήσεις-Παγίδες

Ερώτηση 31: Μήπως το συντριπτικό 80.1% οφείλεται σε σκόπιμα αδύναμο ("χαζό") προγραμματισμό του Heuristic Agent;

Υποδειγματική Απάντηση:
Αυτή είναι η πιο συχνή και επικίνδυνη ερώτηση-παγίδα!
Η απάντηση είναι **κατηγορηματικά ΟΧΙ**: Ο Heuristic Agent σχεδιάστηκε να είναι ένας **εξαιρετικά ισχυρός, ορθολογικός παίκτης**. Ακολουθεί 4 επίπεδα λογικής: αγοράζει άμεσα κάρτες με πόντους, παίρνει τη φθηνότερη κάρτα για να χτίσει μπόνους, δεσμεύει κάρτες και μαζεύει μάρκες με ποικιλία. Για έναν μέσο άνθρωπο παίκτη, ο Heuristic Agent είναι ταχύτατος και εξαιρετικά δύσκολος αντίπαλος!
Ο MCTS τον κερδίζει με 80.1% **όχι επειδή ο Heuristic παίζει άσχημα, αλλά επειδή ο Heuristic είναι απόλυτα προβλέψιμος και μυωπικός**. Ο MCTS εκμεταλλεύεται την απληστία του μέσω των rollouts και του μπλοκάρει τις κρίσιμες κάρτες.

Ερώτηση 32: Είναι το μέγεθος των 251 παρτίδων στατιστικά επαρκές για να τεκμηριώσει την υπεροχή του MCTS;

Υποδειγματική Απάντηση:
Ναι, είναι απόλυτα επαρκές. Εφαρμόζοντας **Διωνυμικό Έλεγχο Σημαντικότητας (Binomial Hypothesis Test)**: * Μηδενική Υπόθεση (H_0): Οι δύο πράκτορες είναι ισοδύναμοι ($p = 0.50$). * Εναλλακτική Υπόθεση (H_1): Ο MCTS υπερέχει ($p > 0.50$). * Αποτέλεσμα: 201 επιτυχίες σε 251 δοκιμές.
Το υπολογιζόμενο p-value είναι:

> $$p\text{-value} < 10^{-12} \ll 0.05$$

Η πιθανότητα το 80.1% να οφείλεται σε τυχαία σύμπτωση είναι μικρότερη από 1 στο ένα τρισεκατομμύριο! Τα αποτελέσματα είναι **ακλόνητα στατιστικά σημαντικά**.

Ερώτηση 33: Τι ακριβώς αποδεικνύει το γράφημα Rolling Win Rate (Σχήμα 24);

Υποδειγματική Απάντηση:
Το γράφημα χρησιμοποιεί παράθυρο κινητού μέσου όρου 10 παρτίδων. Αποδεικνύει ότι: * Στην αρχή (παρτίδες 1–25), όπου η Βάση Γνώσης ήταν άδεια, το win rate κυμαινόταν γύρω στο 65%. * Μετά την 35η παρτίδα, η καμπύλη **σταθεροποιείται σταθερά πάνω από το** 85% έως $95\%+$.
Αυτό αποδεικνύει εμπειρικά ότι **ο αλγόριθμος μαθαίνει μέσω της μόνιμης μνήμης (Transfer Learning)** και βελτιώνει συστηματικά την απόδοσή του!

Ερώτηση 34: Τι αποκαλύπτει η καμπύλη της Βάσης Γνώσης (Σχήμα 25);

Υποδειγματική Απάντηση:
Η καμπύλη ξεκινά με εκθετική άνοδο και σταδιακά σχηματίζει ένα **Λογαριθμικό Πλατώ (Plateau)** γύρω στις **79.808 καταστάσεις**. Αυτό δείχνει ότι οι βασικές διατάξεις του ταμπλό (κυρίως τα ανοίγματα και οι μέσες θέσεις) έχουν πλέον χαρτογραφηθεί, σηματοδοτώντας τη στατιστική σύγκλιση της διαδικασίας μάθησης.

Ερώτηση 35: Τι είναι το "Turning Point" στο γράφημα Battle of Confidence (Σχήμα 20);

Υποδειγματική Απάντηση:
Είναι το κομβικό σημείο στον **Γύρο 18** της παρτίδας 114, όπου η πιθανότητα νίκης του MCTS εκτοξεύεται απότομα από το 50% στο $> 85\%$. Αυτό συμβαίνει διότι στον γύρο αυτό ολοκληρώνεται η "μηχανή" του (Engine Building). Παρότι στον πίνακα σκορ οι παίκτες μπορεί να είναι ισόπαλοι, ο MCTS "βλέπει" μέσω των rollouts ότι οι μόνιμες εκπτώσεις του του εξασφαλίζουν μαθηματικά τη νίκη στους επόμενους γύρους.

Ερώτηση 36: Τι αποδεικνύει το γράφημα Strategy DNA (Σχήμα 19);

Υποδειγματική Απάντηση:
Αποδεικνύει τη θεμελιώδη στρατηγική διαφορά: * Ο Heuristic Agent αγοράζει σχεδόν αποκλειστικά κάρτες Tier 1 (φθηνές, 0 πόντων). * Ο MCTS κατανέμει τις αγορές του στα Tiers 2 και 3 (υψηλών πόντων) και στους Ευγενείς.
Είναι η οπτική απόδειξη της **Αναβαλλόμενης Ικανοποίησης**: ο MCTS απορρίπτει συνειδητά το φθηνό, άμεσο κέρδος προς όφελος της μακροπρόθεσμης υπεροχής.

Ερώτηση 37: Γιατί ο Heuristic Agent κατάφερε να κερδίσει το 19.9% των παρτίδων (50 νίκες); Πότε χάνει ο MCTS;

Υποδειγματική Απάντηση:
Ο MCTS έχασε στις εξής δύο περιπτώσεις: 1. **Ακραία Στοχαστική Εύνοια (Deck Luck):** Όταν η τράπουλα έβγαζε συνεχόμενα κάρτες που ταίριαζαν απόλυτα στις μάρκες που έτυχε να κρατάει ο Heuristic Agent, επιτρέποντάς του να σκοράρει ταχύτατα πριν προλάβει ο MCTS να στήσει τη μηχανή του. 2. **Στενότητα Χρόνου στο Mid-game:** Σε παρτίδες με τεράστιο branching factor (> 40), ο MCTS λόγω του αυστηρού time limit δεν πρόλαβε να κάνει επαρκή rollouts για να εντοπίσει το μπλοκάρισμα.

Ερώτηση 38: Τι συμπεραίνουμε από τη διασπορά στο γράφημα Game Efficiency (Σχήμα 26);

Υποδειγματική Απάντηση:
Το γράφημα δείχνει ότι οι νίκες του MCTS ολοκληρώνονται σταθερά σε **35 έως 48 γύρους** (πολύ μικρή διασπορά). Αντίθετα, οι νίκες του Heuristic Agent φτάνουν μέχρι και τους 70 γύρους (τεράστια διασπορά).

Αυτό αποδεικνύει ότι ο MCTS επιβάλλει τον **έλεγχο του ρυθμού (Tempo Control)** και βρίσκει το συντομότερο μονοπάτι προς τη νίκη, ενώ ο Heuristic κερδίζει μόνο παρατεταμένες παρτίδες φθοράς.

Ερώτηση 39: Πώς πραγματοποιεί ο MCTS "Adversarial Blocking" χωρίς να έχετε γράψει ρητό κώδικα για μπλοκάρισμα;

Υποδειγματική Απάντηση:

Αποτελεί **αναδυόμενη συμπεριφορά (Emergent Behavior)** του δέντρου αναζήτησης!

Στο στάδιο της προσομοίωσης, όταν ο αντίπαλος αγοράζει μία κρίσιμη κάρτα, οι προσομοιώσεις καταλήγουν σε ήττα του MCTS (σκορ 0.0). Αντίθετα, στα κλαδιά όπου ο MCTS επιλέγει `RESERVE_CARD` στη συγκεκριμένη κάρτα, ο αντίπαλος μένει στάσιμος και ο MCTS κερδίζει (σκορ 1.0). Ο αλγόριθμος επιλέγει το reserve καθαρά επειδή του μεγιστοποιεί το UCB1 σκορ, ανακαλύπτοντας το μπλοκάρισμα αυτόνομα!

Ερώτηση 40: Πώς γνωρίζετε ότι ο MCTS δεν έκανε Overfitting στη συγκεκριμένη τράπουλα καρτών;

Υποδειγματική Απάντηση:

Σε κάθε μία από τις 251 παρτίδες, **η τράπουλα ανακατευόταν πλήρως και τυχαία** στην αρχή του αγώνα. Οι 90 κάρτες του Splendor εμφανίζονταν με εντελώς διαφορετική σειρά κάθε φορά. Επομένως, το 80.1% ποσοστό επιτυχίας επιτεύχθηκε σε 251 διαφορετικές συνθήκες ταμπλό, αποκλείοντας κάθε πιθανότητα overfitting.

ΕΝΟΤΗΤΑ 5: Αυτοκριτική, Περιορισμοί & Μελλοντική Έρευνα

Ερώτηση 41: Ποιοι είναι οι τρεις κυριότεροι περιορισμοί της εργασίας σας;

Υποδειγματική Απάντηση:
1. **Το Python GIL:** Εμπόδισε την αξιοποίηση πολλαπλών νημάτων CPU για παράλληλα rollouts. 2. **Η Κλιμάκωση της Βάσης Γνώσης (Flat JSON):** Ενώ λειτούργησε άψογα για 251 παρτίδες (~90MB), για εκατομμύρια παρτίδες θα απαιτούσε εξειδικευμένη NoSQL βάση (π.χ. Redis) λόγω I/O latency. 3. **Η Απουσία Ανθρώπινου Αντιπάλου:** Οι δοκιμές έγιναν έναντι συνθετικού πράκτορα και όχι ανθρώπων, οι οποίοι συχνά παίζουν με ψυχολογική μπλόφα ή απρόβλεπτα λάθη.

Ερώτηση 42: Πώς ακριβώς θα σχεδιάζατε την επέκταση του συστήματος σε αρχιτεκτονική AlphaZero;

Υποδειγματική Απάντηση:
Θα υλοποιούσαμε: 1. Ένα Συνελικτικό ή Residual Νευρωνικό Δίκτυο (ResNet) δύο κεφαλών: * **Policy Head:** Εξάγει διάνυσμα πιθανοτήτων πάνω σε όλες τις νόμιμες κινήσεις, αντικαθιστώντας το Action Pruning. * **Value Head:** Εξάγει απευθείας έναν αριθμό $v \in [-1, 1]$ που εκτιμά την αξία του ταμπλό, **καταργώντας εντελώς τα αργά rollouts**! 2. Αυτόνομη εκπαίδευση μέσω **Self-Play Reinforcement Learning**, όπου ο πράκτορας παίζει εκατομμύρια παρτίδες εναντίον του εαυτού του, βελτιώνοντας τα βάρη του μέσω Stochastic Gradient Descent.

Ερώτηση 43: Ποιες μαθηματικές και θεωρητικές προκλήσεις εισάγει η επέκταση σε 3 ή 4 παίκτες;

Υποδειγματική Απάντηση:
Σε παιχνίδια > 2 παικτών καταρρέει η ιδιότητα του αυστηρού Zero-Sum. Εμφανίζονται δύο τεράστια προβλήματα: 1. **Το Φαινόμενο Kingmaking:** Ένας παίκτης που δεν έχει ελπίδα νίκης μπορεί με την κίνησή του να καθορίσει ποιος από τους άλλους δύο θα κερδίσει. 2. **Αλγοριθμική Προσαρμογή:** Ο κλασικός Minimax/MCTS πρέπει να αντικατασταθεί από τον αλγόριθμο Max^n (όπου κάθε κόμβος διατηρεί διάνυσμα κερδών για όλους τους παίκτες) ή από υποθέσεις **Paranoid Search** (όπου υποθέτουμε ότι όλοι οι άλλοι παίκτες έχουν συμμαχήσει εναντίον μας).

Ερώτηση 44: Αν είχατε άλλους 6 μήνες στη διάθεσή σας, ποια θα ήταν η πρώτη σας προτεραιότητα;

Υποδειγματική Απάντηση:
Η πρώτη προτεραιότητα θα ήταν η **επανυλοποίηση της μηχανής του Splendor σε C++20 με pybind11 bindings**. Αυτό θα υπερέβαινε το GIL, επιτρέποντας multi-threaded root parallelization σε 16 πυρήνες CPU.

Με 20.000+ rollouts ανά κίνηση, θα μπορούσαμε να εφαρμόσουμε Self-Play και να εκπαιδεύσουμε ένα πραγματικό Νευρωνικό Δίκτυο επιπέδου AlphaZero!

Ερώτηση 45: Ποιο είναι το τελικό, κεντρικό συμπέρασμα της διπλωματικής σας εργασίας σε μία μόνο πρόταση;

Υποδειγματική Απάντηση:

«Η παρούσα διπλωματική εργασία απέδειξε ότι ο αλγόριθμος Monte Carlo Tree Search, ενισχυμένος με στοχευμένη αποκοπή ενεργειών (Action Pruning), δυναμική προσαρμογή εξερεύνησης (Adaptive UCT) και μόνιμη βάση γνώσης, υπερέχει συντριπτικά έναντι των παραδοσιακών άπληστων ευρετικών μεθόδων, κατακτώντας ποσοστό νικών 80.1% και ανακαλύπτοντας αυτόνομα τη στρατηγική της αναβαλλόμενης ικανοποίησης (Engine Building) σε πολύπλοκα περιβάλλοντα διαχείρισης πόρων.»

Τέλος Εγχειριδίου Ερωτήσεων & Απαντήσεων.